

PinProbe A1控制箱 嵌入式整体方案与架构报告

单机控制主线 · RamVector 数据总线 · 状态机安全逻辑 · 多机扩展路线

STM32F103RCT6 / FreeRTOS / SCPI + Modbus RTU | 2026 年 6 月

目 录

Part 1 项目定位

P3	项目目标与边界
P4	总体方案与运行链路
P5	软件框架与任务分工

Part 2 核心架构

P6	RamVector 黑板式数据总线
P7	状态机与安全控制策略
P8	IO 监视反馈闭环
P9	通讯、诊断与持久化
P10	AppLog 结构化诊断黑匣子

Part 3 价值评估

P11	核心优势总结
P12	当前加固清单

Part 4 演进路线

P13	后续扩展方向
P14	最终结论

项目目标与边界

当前阶段：单机箱体控制固件 | 后续阶段：CAN 多机协同

项目定位

- > 控制 PinProbe A1 箱体的气缸、锁、LED、传感器和安全输入
- > 对外提供 SCPI 调试/控制接口，内部通过 RS485 控制 BSM IO 板
- > 用 FreeRTOS 将通讯、状态机、看门狗和诊断任务分离
- > 以 RamVector 作为唯一状态与命令交换中心

当前已实现

- > 25ms 系统节拍驱动 ModBusTask 与 StateVectorTask
- > SCPI 命令写入 RamVector 命令槽，状态查询读取 IO 镜像
- > RS485 故障保护、门限位消抖、急停/激光防夹、风险模式
- > Flash 配置、POST 自检、固件版本/运行时间/日志查询

明确不混入当前阶段

- > CAN 多机同步仍属于后续阶段，不视为单机已落地功能
- > W25Q128 快照、OTA、跨节点仲裁按多机方案统一规划
- > 波特率配置是出厂/维护动作，不作为运行期控制链路
- > 旧 StateMachine.c 已废弃，只保留历史参考

当前主线目标：把单机箱体控制做成 **低耦合、可诊断、可扩展** 的工业控制内核

总体方案与运行链路

从上位机命令到物理 IO 的闭环路径 | 命令与观测解耦



链路段	设计选择	工程意义
控制入口	SCPI 只负责参数解析和命令发布	不会直接碰 RS485，避免和周期 IO 任务争总线
数据交换	RamVector 存命令槽、IO 镜像、全局状态和节点状态	模块之间通过结构化内存通信，降低调用耦合
执行出口	ModBusTask 独占 RS485，并在同一任务内执行命令	避免多任务同时操作 USART3，保持总线行为可预测
反馈闭环	状态查询读 RamVector IO 镜像，不发起现场 IO 读取	查询轻量、确定，不影响实时控制节拍

软件框架与任务分工

FreeRTOS 任务模型 | 职责分离 | 固定节拍

任务/模块	周期/优先级	职责	关键文件
SCPITask	10ms / Low	处理 USART1 SCPI 命令、输出查询结果、泵送实时日志	main.c, scpi-def.c
ModBusTask	25ms flag / Normal1	读取 BSM IO 输入/输出，更新 RamVector 镜像，执行命令槽	main.c, RS485.c, cmd_exec.c
StateVectorTask	25ms flag / Normal	纯逻辑状态机，读取 IO 镜像并发布锁/气缸/LED 命令	state_vector.c
WatchDogTask	500ms / Low	刷新独立看门狗，避免任务整体失控后长期挂死	freertos.c, iwdg.c
defaultTask	1s / Normal	当前为集群健康监测占位，多机阶段扫描节点心跳	freertos.c
SysTimer	25ms	统一唤醒 ModBusTask 与 StateVectorTask	freertos.c

25ms
控制节拍

3
命令通道

1024B
RamVector

76
日志容量

RamVector 黑板式数据总线

固定 1024B 内存布局 | 单机可用，多机预留 | 命令/状态/IO 解耦

区域	内容
帧头	magic、version、timestamp、crc32，为多机同步预留
命令区	lock、cylinder、led 三个独立 CmdSlot，带优先级
路由区	node_mask、source_node、ack_flags，多机阶段启用
状态区	global_state、error_code、node_status[8]
IO 镜像	io_state[8]，单机只用 index 0
扩展区	ext_params[786]，给后续复杂参数和版本扩展留余量

架构价值

- > SCPI、状态机、执行层都围绕同一份数据表协作
- > 命令是意图，IO 镜像是事实，状态是推导结果
- > 单机阶段不需要复杂队列即可获得清晰模块边界
- > 多机阶段可以直接把同一张表扩展为 CAN 同步载体

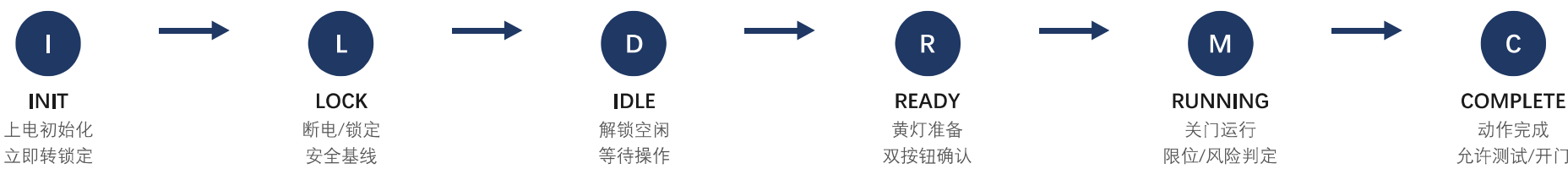
已识别加固点

- > IO 镜像读取需要整帧快照，避免字段半新半旧
- > 命令槽需要原子 Take/Clear，避免新命令被 Clear 覆盖
- > 执行命令仍保持在临界区外，避免阻塞实时路径

RamVector 是当前项目的核心边界：**外部命令、现场 IO、系统状态** 都在这里汇合

状态机与安全控制策略

时间触发状态机 | IO 观测纠偏 | 安全事件优先



安全机制	当前策略	价值
断电纠偏	power_out 断开时，非紧急/非初始化状态无条件回 LOCK	物理断电优先于软件状态，避免状态假象
按钮确认	电源长按 300ms，关门 Ready 后双按钮 500ms，开门单按钮 200ms	降低误触发和夹手机率
防夹保护	关门后段激光触发进入 EMERGENCY，红灯、锁定、开门释放	安全事件直接高优先级注入命令槽
RS485 故障	连续失败 10 次后黄灯告警并停止自动操作，恢复后自动清除	IO 不可靠时拒绝继续做自动判断
风险模式	限位异常场景下可用气压与学习时间辅助判定关门完成	现场传感器退化时提供受控兜底路径

IO 监视反馈闭环

状态机不盲信命令结果 | 每轮通过现场 IO 镜像纠偏



反馈环节	数据来源	判断/动作	工程意义
输入监视	raw_in_lo / raw_in_hi	读取限位、按钮、急停、激光、气压，并对限位/按钮做 3 次采样消抖	把机械触点抖动隔离在状态机入口
输出反馈	raw_out_lo / raw_out_hi	通过气缸、锁、LED、电源输出位判断动作是否已经实际生效	状态不是单纯软件变量，而是被 IO 输出反馈校正
通讯可信度	rs485_ok	IO 读取失败时跳过自动判断；连续失败达到阈值后黄灯告警	现场数据不可靠时不继续做危险判断
状态纠偏	输入 + 输出组合	power_out 断开回 LOCK，门下限位进入 COMPLETE，关门输出生效进入 RUNNING	物理事实优先于历史软件状态
动作闭环	命令槽 + 下一轮输出读取	本轮写命令，后续通过输出镜像确认动作路径	避免“发了命令就认为完成”的开环控制

核心逻辑：命令只是意图，IO 反馈才是事实

通讯、诊断与持久化

控制入口、现场总线、运行记录、配置存储四条线分离

能力	实现	说明
SCPI 控制台	USART1 + libscpi port	控制气缸/锁/LED，读取系统状态、IO、日志、版本、运行时间
BSM IO 通讯	USART3 RS485 + Modbus 功能码 01/02/03/05	周期采样输入/输出，写单线圈执行动作
DMA 接收	UART DMA 双缓冲 + IDLE 中断	降低串口接收阻塞，提高帧边界处理确定性
AppLog	独立 RAM 日志区 0x2000A400	记录状态、动作、事件、IO，可实时 UART 输出或 SCPI 拉取
Flash 配置	内部 Flash 最后一页 2KB	保存 IDN、急停类型、风险模式、波特率等配置，带 CRC 校验

上电初始化链路

- > HAL、时钟、GPIO/DMA/UART/SPI/CAN/TIM/IWDG 初始化
- > 开启 UART IDLE 和 DMA 中断，启动 USART1/USART3 DMA 接收
- > Flash_Init 加载配置，SCPI_Init 建立命令上下文
- > AppLog_Init 后执行 POST：RCC、Flash CRC、RS485、IO板型号
- > 启动 FreeRTOS 调度器，由 SysTimer 驱动周期任务

诊断策略

项目没有把诊断混入 RamVector，而是用独立 AppLog 记录“发生了什么”。RamVector 负责实时状态，AppLog 负责事后解释，两者边界清晰。

AppLog 结构化诊断黑匣子

独立于 RamVector 的诊断面 | 固定 RAM 区 | SCPI 拉取 + 实时 UART

设计点	实现方式	价值
独立内存段	scatter 固定 RW_APPLOG 0x2000A400 ，大小 0x500	诊断数据不占用 RamVector，避免污染实时状态表
固定记录格式	每条 16B ，含 tick、seq、node、level、module、event、arg	比 printf 字符串更省 RAM，也便于后续落盘或上传
环形缓冲	容量 76 条，满后覆盖最旧记录并累计 drop_count	不会因日志堆积阻塞控制链路
双消费通道	SCPI read_index 与实时 realtime_index 分离	人工查询和实时串口输出互不抢同一个游标
文本格式化	读取时再格式化为 STATE/ACTION/EVENT/IO 文本	运行期存结构体，输出期再转换，节省现场 RAM

当前记录内容

- > 状态跳转：LOCK、IDLE、READY、RUNNING、EMERGENCY、COMPLETE
- > 动作耗时：关门开始/完成、开门开始/完成、阶段里程碑
- > 安全事件：RS485 故障、急停、激光防夹、风险模式、气压异常
- > 命令来源：SCPI 锁、气缸、LED 操作，以及按钮触发
- > IO 快照：按需记录输入/输出原始字节变化

访问方式

- > **READ:LOG:NEXT?** 拉取下一条日志
- > **READ:LOG:ALL?** 拉取当前缓存全部日志
- > **READ:LOG:STATus?** 查看容量、待读数、丢弃计数、实时开关
- > **CONFigure:LOG:UART** 开关实时 UART 输出
- > UART 忙时实时输出直接跳过，不等待串口空闲

架构定位

RamVector 回答“系统现在是什么状态”，AppLog 回答“系统为什么变成这个状态”。它是单机调试、现场验收、异常复盘和后续 Flash 故障日志的共同基础。

注：实时 UART 是 best-effort 诊断输出，不承诺全量；需要完整读取时以 SCPI 拉取通道和 drop_count 为准。

核心优势总结

当前实现的工程价值 | 面向现场可靠运行

1. 低耦合控制内核

- > SCPI、状态机、RS485 执行层不直接互相驱动
- > RamVector 作为唯一共享数据平面
- > 命令、事实、状态分区清楚，便于审查和扩展

2. 确定性时序

- > 25ms 统一节拍驱动关键控制路径
- > ModBusTask 独占现场总线，避免串口竞争
- > 状态机不直接发 RS485，逻辑执行时间稳定

3. 安全优先级明确

- > 急停、激光、RS485 故障使用更高命令优先级
- > 断电回 LOCK、按钮确认、限位消抖形成多层保护
- > 风险模式可配置，不把兜底逻辑写死

4. 现场可诊断

- > POST 输出启动自检结果与复位原因
- > AppLog 记录状态跳转、动作耗时、关键事件
- > SCPI 可读取版本、运行时间、IO、日志状态

5. 配置可持久化

- > Flash 配置带魔数、版本和 CRC 校验
- > IDN、急停类型、风险模式可通过 SCPI 配置
- > 配置失效时可回默认值，不阻塞固件启动

6. 多机预留充分

- > RamVector 已按 8 节点布局预留 node_status 和 io_state
- > CAN、SPI 外设初始化已在工程中存在
- > defaultTask 已留出集群健康监控位置

当前加固清单

不是重构方向，而是上线前的边界收紧

优先级	事项	原因	涉及文件
P0	气缸双线圈写入顺序加固	当前先合目标线圈再断对向线圈，存在短暂双输出窗口；应改为先断对向，再合目标	BsmRelay.c
P1	RamVector IO 镜像整帧快照	状态机和 SCPI 查询读取局部副本，避免结构体字段半新半旧	ram_vector.c/h, state_vector.c, scpi-def.c
P1	命令槽原子 Take/Clear	只保证“取命令并清槽”是原子事务，不扩展成队列，不把执行放进临界区	ram_vector.c/h, cmd_exec.c
P2	WriteIO 返回值汇总检查	不做重试、不增加等待；只记录或上报失败，提升可诊断性	BsmRelay.c, cmd_exec.c
P2	ReadIO 发送失败直接返回	读命令发送失败时无需继续等待 50ms 响应，可减少无效超时并改善诊断	RS485.c, BsmRelay.c
P4	清理废弃旧状态机接口	降低维护干扰，避免后续误用 StateMachine.c 和旧 Status 函数	BsmRelay.c/h, StateMachine.c/h

加固原则

当前架构不需要大改。优先处理执行器安全、共享数据一致性、失败可诊断三类问题，避免引入会改变现场时序的大规模抽象。

后续扩展方向

按阶段推进：单机加固 → 产线诊断 → 多机协同 → 远程维护

阶段	目标	主要内容	收益
阶段 1	单机上线加固	完成 P0/P1 事项：气缸写入顺序、IO 快照、命令槽 Take/Clear、WriteIO/ReadIO 诊断	降低现场偶发风险
阶段 2	产线测试模式	增加一键 IO/传感器/通讯自测，输出标准化验收结果，补充 SCPI 错误反馈覆盖	缩短装配和维修定位时间
阶段 3	运行数据沉淀	将关键故障、动作计数、设备寿命累计迁移到 W25Q128 或备份区	支持预防性维护
阶段 4	CAN 多机协同	启用 node_id、node_status、heartbeat、version、CRC、ack_flags，完成 1~8 节点同步	从单机控制升级为集群控制
阶段 5	PLC 标准化接入	对外主接口逐步从 SCPI 调试转向 Modbus RTU 从站，SCPI 保留为维护控制台	更贴近产线集成
阶段 6	OTA 与快照恢复	W25Q128 分区：固件暂存、向量快照、故障日志、配置区；支持断点恢复	降低维护成本

先加固单机 再沉淀诊断 后启用多机 最后做远程维护

最终结论

当前项目的技术判断 | 保持主架构，做边界加固

RamVector 作为主数据面 ModBusTask 独占现场总线 StateVector 保持纯逻辑 SCPI 保持调试入口

总体判断

PinProbe A1 当前单机固件已经形成清晰的工业嵌入式控制框架：上位机命令、现场 IO、状态机、安全事件、配置与诊断分别有明确边界。项目的核心价值不在复杂抽象，而在用较少模块实现了可预测的控制闭环。

保留

- > 25ms 时间触发主节拍
- > RamVector 命令/状态/IO 模型
- > 状态机按物理语义顺序展开

加固

- > 执行器写入顺序
- > 共享数据快照一致性
- > 命令取清原子性和失败诊断

扩展

- > 出厂测试与验收自动化
- > W25Q128 日志/快照/OTA
- > CAN 反射内存式多机协同

推荐路线：不重构主架构，沿当前边界逐项加固，再进入多机阶段